```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# Step 1: Load the dataset
data = load_iris()
X = data.data
y = data.target

# Step 2: Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 3: Create the neural network
model = MLPClassifier(
    hidden_layer_sizes=(10,),
    max_iter=1000,
    random_state=42
)

# Step 4: Train the network using backpropagation
model.fit(X_train, y_train)

# Step 5: Predict test data
y_pred = model.predict(X_test)
```

```
# Step 6: Evaluate performance
accuracy = accuracy_score(y_test, y_pred)

print("Actual Values:", y_test)
print("Predicted Values:", y_pred)
print("Accuracy:", round(accuracy * 100, 2), "%")
```



```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, classification_report

# Step 1: Load the dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Step 2: Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Step 3: Create a weak classifier
weak_model = DecisionTreeClassifier(
    max_depth=1, random_state=42
```

```
)

# Step 4: Train the individual weak classifier
weak_model.fit(X_train, y_train)
weak_pred = weak_model.predict(X_test)

weak_accuracy = accuracy_score(y_test, weak_pred)

# Step 5: Create the AdaBoost ensemble
ada_model = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=50,
    learning_rate=1.0,
    random_state=42
)

# Step 6: Train the ensemble
ada_model.fit(X_train, y_train)
ada_pred = ada_model.predict(X_test)

ada_accuracy = accuracy_score(y_test, ada_pred)

# Step 7: Compare performance
print("Individual Weak Classifier Accuracy:",
      round(weak_accuracy * 100, 2), "%")

print("AdaBoost Ensemble Accuracy:",
      round(ada_accuracy * 100, 2), "%")
```

```
print("\nAdaBoost Classification Report:")
print(classification_report(
   y_test, ada_pred, target_names=data.target_names
))

# Step 8: Display comparison
print("\nPerformance Comparison")
print("----------------------")
print(f"Weak Classifier : {weak_accuracy * 100:.2f}%")
print(f"AdaBoost       : {ada_accuracy * 100:.2f}%")
```



Naive Bayes Classifier using Iris Dataset

```
# Step 1: Import libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report

# Step 2: Load the Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Step 3: Split the dataset into training and testing data
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 4: Create the Naive Bayes model
model = GaussianNB()

# Step 5: Train the model
model.fit(X_train, y_train)

# Step 6: Predict the flower species
y_pred = model.predict(X_test)

# Step 7: Display accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy * 100, "%")

# Step 8: Display classification report
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred, target_names=iris.target_names
))

# Step 9: Predict a new flower
new_flower = [[5.1, 3.5, 1.4, 0.2]]
prediction = model.predict(new_flower)

print("Predicted Flower:", iris.target_names[prediction[0]])
```

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)
import matplotlib.pyplot as plt


# Load dataset
iris = load_iris()

X = iris.data
y = iris.target


# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
```

```
)



# Feature scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)



# Create K-NN model
model = KNeighborsClassifier(n_neighbors=5)



# Train the model
model.fit(X_train, y_train)



# Predict
y_pred = model.predict(X_test)



# Classification metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')
```

```
# Display results
print("K-NN Classification Results")
print("---------------------------")
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall :", recall)
print("F1 Score :", f1)

print("\nClassification Report:")
print(classification_report(
   y_test,
   y_pred,
   target_names=iris.target_names
))



# Graph
plt.figure(figsize=(8, 6))

plt.scatter(
   X_test[:, 0],
   X_test[:, 1],
   c=y_test,
   s=80
)

plt.xlabel("Sepal Length")
```

```
plt.ylabel("Sepal Width")
plt.title("K-NN Classification - Iris Dataset")

plt.show()
```

```
# Import required libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# 1. Load the Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# 2. Split the dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# 3. Create Decision Tree model
model = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=3,
    random_state=42
)

# 4. Train the Decision Tree
model.fit(X_train, y_train)

# 5. Predict the test data
y_pred = model.predict(X_test)

# 6. Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

# 7. Visualize the Decision Tree
plt.figure(figsize=(15, 8))

plot_tree(
    model,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True
)
```

```
plt.title("Decision Tree - Iris Dataset")
plt.show()
```

[redacted]

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create SVM classifier
model = SVC(kernel='linear')

# Train the model
model.fit(X_train, y_train)

# Predict
```

```
y_pred = model.predict(X_test)


# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))


# Predict a new flower
print("Prediction:", model.predict([[5.1, 3.5, 1.4, 0.2]]))
```

